

Dívida Técnica em Apps Mobile: Análise de Code Smells em Flutter e React Native

Davi Aguilar Nunes Oliveira

Engenharia de Software

Disciplina base do trabalho

Qualidade de Código

Code smells & métricas

Dívida Técnica

Impacto na manutenibilidade

Dev Mobile Multiplataforma

Flutter e React Native

Este trabalho situa-se na intersecção entre qualidade de software e desenvolvimento mobile multiplataforma, investigando como decisões arquiteturais nos frameworks mais populares do mercado influenciam a saúde de longo prazo das aplicações.

Como a arquitetura inerente de cada framework influencia a incidência de Code Smells específicos e, consequentemente, o acúmulo de dívida técnica em projetos mobile?

Flutter

- ▶ Árvore de widgets profunda (deep nesting)
- ▶ Long build methods difíceis de testar
- ▶ State management overkill em telas simples

React Native

- ▶ Large components / God Classes
- ▶ Prop drilling por múltiplos níveis
- ▶ Re-renderings ineficientes por falta de memoização



Mercado em expansão

Flutter e React Native dominam o desenvolvimento mobile multiplataforma. Decisões arquiteturais impactam milhares de produtos.



Custo da dívida técnica

Dívida técnica acumulada aumenta o custo de manutenção e evolução de software em até 4x ao longo do ciclo de vida do produto.



Lacuna na literatura

Poucos estudos comparam sistematicamente code smells entre os dois ecossistemas com métricas quantitativas padronizadas.



Impacto prático

O trabalho gerará diretrizes objetivas para equipes decidirem qual framework minimiza a dívida técnica conforme o tipo de projeto.

OBJETIVO GERAL

Comparar quantitativamente a incidência de code smells e o nível de dívida técnica em projetos Flutter e React Native, identificando padrões arquiteturais que contribuem para cada tipo de smell.

OBJETIVOS ESPECÍFICOS

- 1 Mapear os code smells mais frequentes em repositórios Flutter e React Native open source
- 2 Medir complexidade ciclomática, acoplamento e coesão nos dois ecossistemas
- 3 Analisar se o deep nesting do Flutter gera impacto mensurável superior ao prop drilling do React Native
- 4 Avaliar a correlação entre dependências de terceiros (RN) e aumento de dívida técnica
- 5 Propor um guia de decisão arquitetural para reduzir a dívida técnica no início de novos projetos

01

An empirical study of code smells in JavaScript projects

Palomba et al. (2018) · Proc. 40th IEEE/ACM ICSE · DOI: 10.1145/3180155.3180238

Investigou code smells em aplicações JavaScript server-side, analisando 12 tipos de smells em 537 releases de 5 projetos populares. Estabelece as métricas base (complexidade, fault-proneness) que serão adaptadas neste TCC para o contexto React Native.

Metodologia de detecção de smells adaptada para RN

02

A comparison of native and cross-platform frameworks for mobile applications

Nawrocki et al. (2021) · Computer, IEEE · Vol. 54, n. 3, pp. 18–27 · DOI: 10.1109/MC.2020.2983893

Comparou frameworks nativos e multiplataforma (incluindo Flutter e React Native) em termos de qualidade, experiência do desenvolvedor e manutenibilidade. Base direta para as hipóteses deste trabalho sobre diferenças arquiteturais.

Comparação direta dos dois ecossistemas

FERRAMENTAS

Dart Analyzer / flutter_lints

Análise estática Flutter

ESLint + SonarJS

Análise estática React Native

CodeClimate / SonarQube

Métricas de manutenibilidade

GitHub API

Coleta de repositórios open source

React DevTools Profiler

Detecção de re-renderings

CORPUS DE ANÁLISE

20+

Repositórios Flutter

open source · GitHub

20+

Repositórios React Native

open source · GitHub

3

Métricas principais

Complexidade · Acoplamento · Coesão



ABORDAGEM

